SIFT-based Panorama Image Stitching

Name: Tuguldur (투굴두루) Student ID: 202213827 Course: Computer Vision

1. What I built

For this project I had to take 5 photos that go from left to right and stick them together into one wide panorama. The program reads input_1.jpg through input_5.jpg from its own folder, joins them in order, and saves panorama_result.jpg. There are no fixed file paths, so it runs anywhere as long as the 5 photos sit next to the script. I did not start from zero. Most of it is the code from our three class practices put together, plus a few extra parts to keep it from breaking on real photos.

The whole thing runs in five steps, which I go through below: find matching points between neighbour photos, work out how each photo lines up with the next, bend the photos onto a cylinder so a wide view looks right, blend them so the seams do not show, and crop off the empty black corners at the end.

2. How it works

- **Matching points (Practice 2).** For every pair of side-by-side photos I run SIFT to get keypoints and descriptors, then match them with the two filters from class. The ratio test drops a match unless the best one is clearly closer than the runner-up (I used 0.75), and the cross-check keeps it only if both photos agree. That leaves the good matches, usually a few hundred to over a thousand per pair.
- **Finding the transform (Practice 3).** Next I need to know how one photo lines up with the next. That is a homography, the 3x3 matrix from the transform practice. I fit it with RANSAC so the few bad matches that sneak past the filters get thrown out. I line everything up to the middle photo (number 3) and work outwards to both sides. Doing it from the middle splits the stretching to both ends instead of dumping it all on one side.
- **Bending onto a cylinder.** A homography on its own is fine for a small turn, but across a wide sweep it stretches the end the image size blows up (I actually hit a crash from this early on). So before anything else I bend each photo onto a cylinder, which is closer to how you really shoot a panorama by turning around. The cylinder needs a focal length, which I do not really know for a phone, so I tried a few (24mm, 35mm, 50mm) and went with whatever looked best. The bending uses backward warping from Practice 3: for each output pixel I look up where it came from in the input, so there are no holes.
- **Blending (the fade idea from Practice 1).** If I just drop the photos on top of each other you can see hard lines where they meet, mostly in the smooth sky. To hide that I give each photo a weight that is high in the middle and fades to nothing at the edges, then take a weighted average where they overlap. It is the same fade idea as the Gaussian work in Practice 1, and it makes the joins disappear instead of showing a seam.
- **Cropping.** After warping there are slanted black gaps around the edge. My crop keeps the full width and shaves the top and bottom by just enough to clear the black, so I get a clean rectangle that still looks wide.

3. Results

I tested the program twice. First on 5 photos I took myself around campus, then on the official test images. Here is the campus one:



Figure 1. My own campus test (50mm).

The road, crosswalk, trees and buildings all join up, and the brightness stays even across the whole thing. 50mm looked the most natural here. The sweep was pretty wide, so the plain planar mode blew up and the cylinder was really doing the work.

And here is the result on the given test images:



Figure 2. Official test images (35mm, fullwidth crop).

Everything lines up across the joins and the dusk light blends smoothly. This time 35mm looked better than 50mm, because the scene spreads out more sideways and the flatter setting suited it. I've tested all modes on my code. And 35mm, fullwidth crop feels like more panorama image.

4. Trying different focal lengths

Since I do not know the real focal length, I made the program able to build the panorama at 24mm, 35mm and 50mm and just compared them. The smaller the number, the more the cylinder bends the picture; the bigger the number, the flatter it gets. Below is 24mm against 35mm on the test images. 24mm bends the buildings and the horizon too much and feels like a fisheye lens, while 35mm keeps things looking straight, so I picked 35mm.



Figure 3. 24mm (top) is too curved; 35mm (bottom) looks natural.

5. Trying different crops

I also tried four ways of cropping the black corners. "keep_all" keeps everything but leaves big black wedges. "tight" finds the largest rectangle with no black at all, but on a curved panorama that throws away almost the whole picture. "relaxed" allows a little black to keep more of the image. "fullwidth" keeps the full width and only trims the top and bottom, which gave the cleanest wide result, so that is what I used.



Figure 4. Four crop options at 35mm. Fullwidth (bottom-right) is the one I chose.

6. Same comparisons on my campus photos

Just to show the same tests on my own photos (campus picked 50mm):



Figure 5. Campus focal test: 24mm vs 50mm.

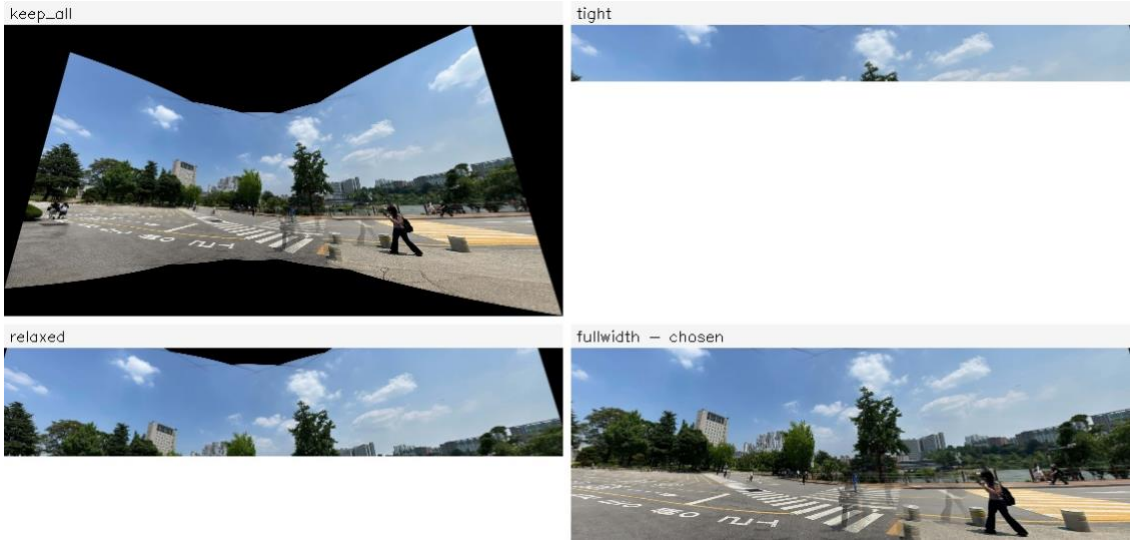


Figure 6. Campus crop options (50mm). Fullwidth bottom-right.

7. Comparing the two test sets

The two sets did not want the same setting: campus looked best at 50mm and the official set at 35mm. There is no single focal length that is always right, which is exactly why we were allowed to try a few and keep the best. Both share the same weak spot: the far right edge is a bit soft and bent, since it is furthest from the middle photo and gets stretched the most. My campus shot also has more ghosts of moving people, while the emptier test plaza came out cleaner.

8. Design choices and limitations

- **Cylinder and homography together.** The homography figures itself out from the matches, so I do not measure the camera, but on a wide sweep it goes wild. The cylinder keeps the shape under control, so I use both: cylinder first, then homography.
- **A simpler transform was worse.** I tried swapping the homography for a simpler affine transform to flatten the bend on the right. It flattened the shape but made everything blurry and doubled, because handheld shots have angle differences only a full homography can line up. So I kept the homography.
- **Not crashing.** A bad match can make a broken transform that asks for a giant image and crashes. To stop that, I check each transform is sane before using it and cap the output size, and if the chosen setting fails the program tries the others instead of dying.

9. Conclusion

Gluing the three practices together (SIFT matching, homography with RANSAC, and warping with blending) was enough to make a panorama program that works on real phone photos. The main thing I learned is that the class math is correct, but real photos need a few guard rails like the cylinder warp and the size cap or it falls over. The final program reads the 5 inputs on its own and saves one wide panorama with no manual steps.